ASSIGNMENT-11.2

NAME: J.SATHWIK

HTNO: 2303A51055

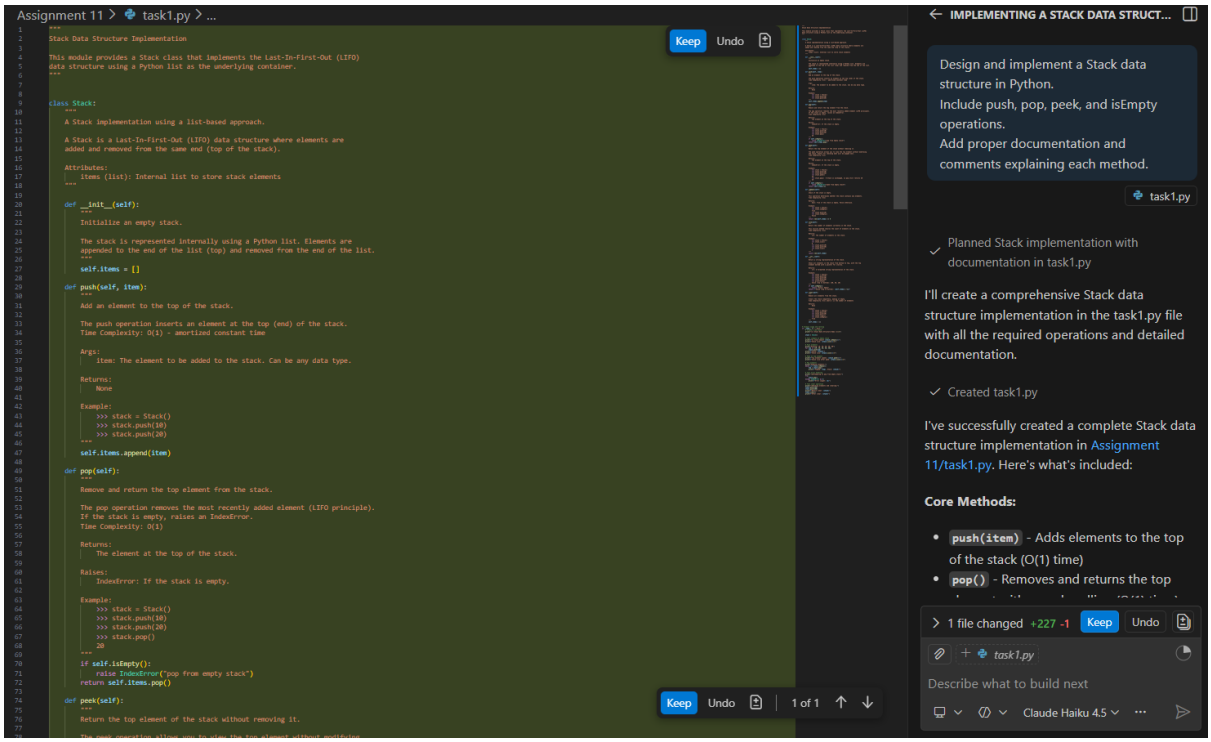
BATCH: 16

Task Description -1 – (Stack Using AI Guidance)

• Task: With the help of AI, design and implement a Stack data structure supporting basic stack operations.

Expected Output:

• A Python Stack class supporting push, pop, peek, and empty-check operations with proper documentation.



OUTPUT:

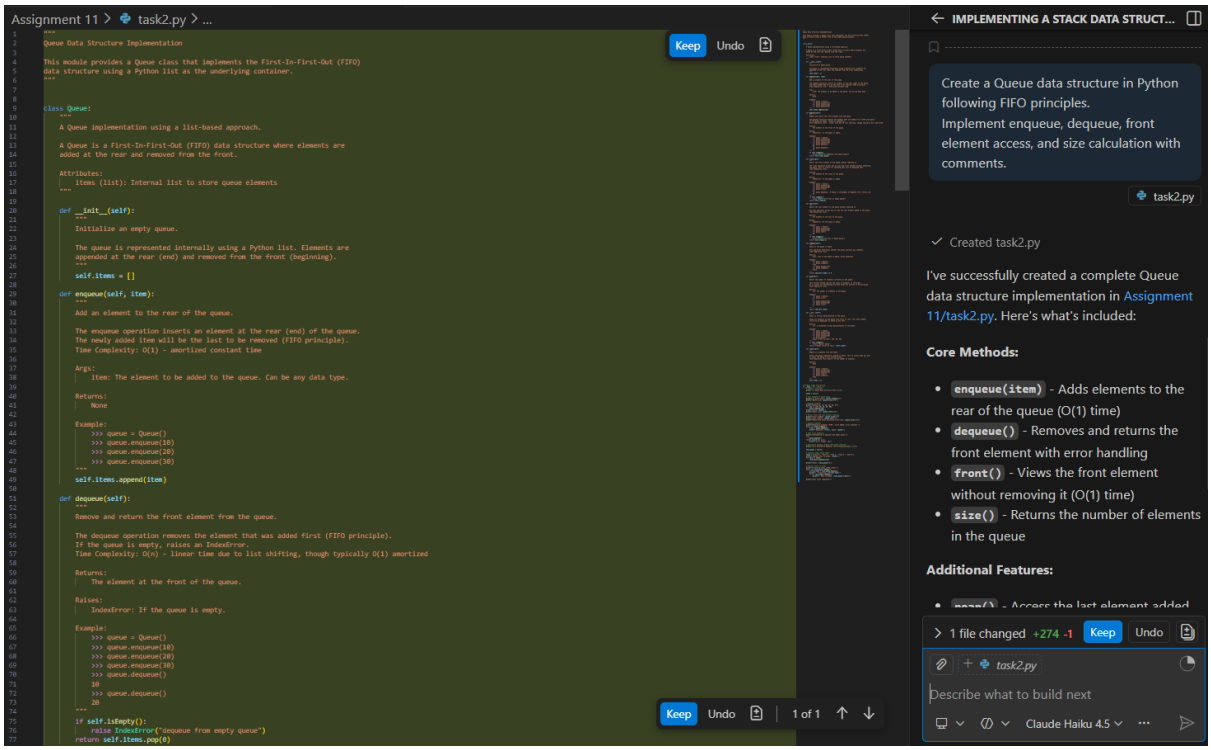
```
Popping elements:
Popped: 50, Stack: Stack (top to bottom): [40, 30, 20, 10]
Popped: 40, Stack: Stack (top to bottom): [30, 20, 10]
Popped: 30, Stack: Stack (top to bottom): [20, 10]
Popped: 20, Stack: Stack (top to bottom): [10]
Popped: 10, Stack: Stack: Empty

Attempting to pop from empty stack:
Error caught: pop from empty stack

Pushing elements and clearing:
Before clear: Stack (top to bottom): [200, 100]
After clear: Stack: Empty
```

Task Description -2 – (Queue Design)

- **Task:Use AI assistance to create a Queue data structure following FIFO principles**

Expected Output:

- **A complete Queue implementation including enqueue, dequeue, front element access, and size calculation**



OUTPUT:

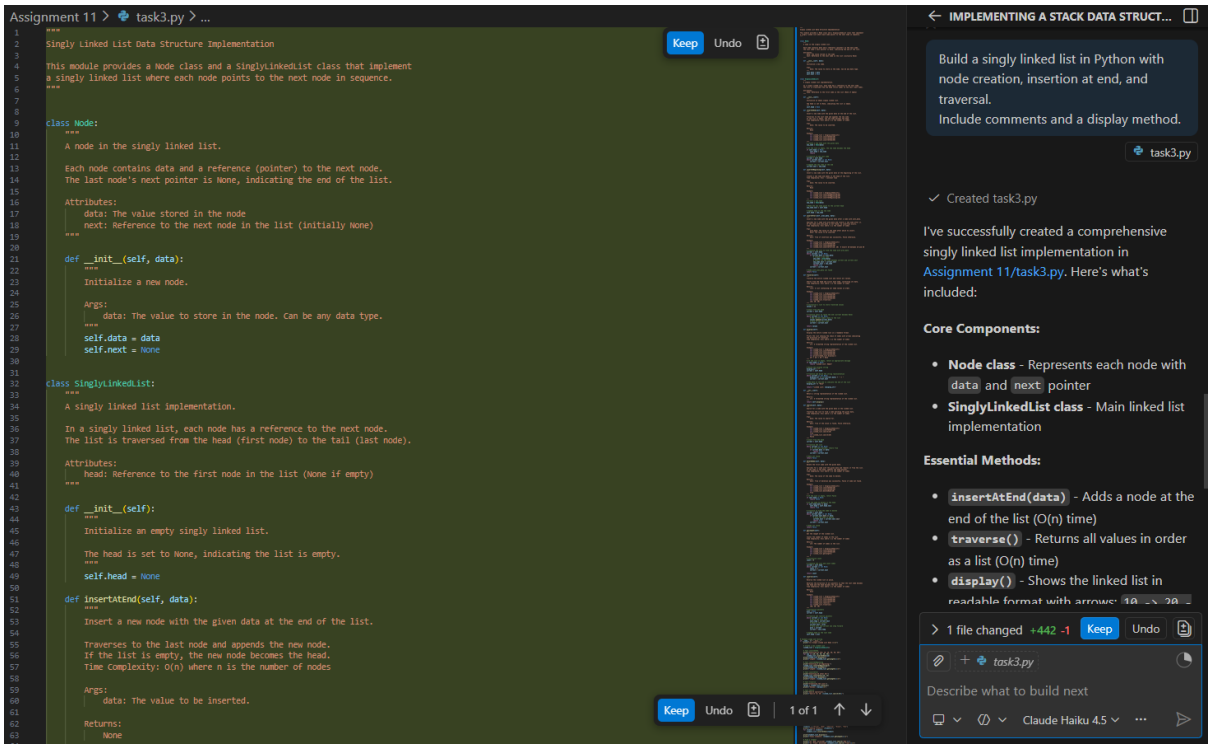
```
Adding tasks to queue: ['Task A', 'Task B', 'Task C', 'Task D', 'Task E']
Queue: Queue (front to rear): ['Task A', 'Task B', 'Task C', 'Task D', 'Task E']

Processing tasks (FIFO order):
  Processing: Task A
  Next in queue: Task B
  Processing: Task B
  Next in queue: Task C
  Processing: Task C
  Next in queue: Task D
  Processing: Task D
  Next in queue: Task E
  Processing: Task E

All tasks completed!
```

Task Description -3 –(Singly Linked List Construction)

• Task: Utilize AI to build a singly linked list supporting insertion and traversal.

Expected Output:

• Correctly functioning linked list with node creation, insertion logic, and display functionality.



OUTPUT:

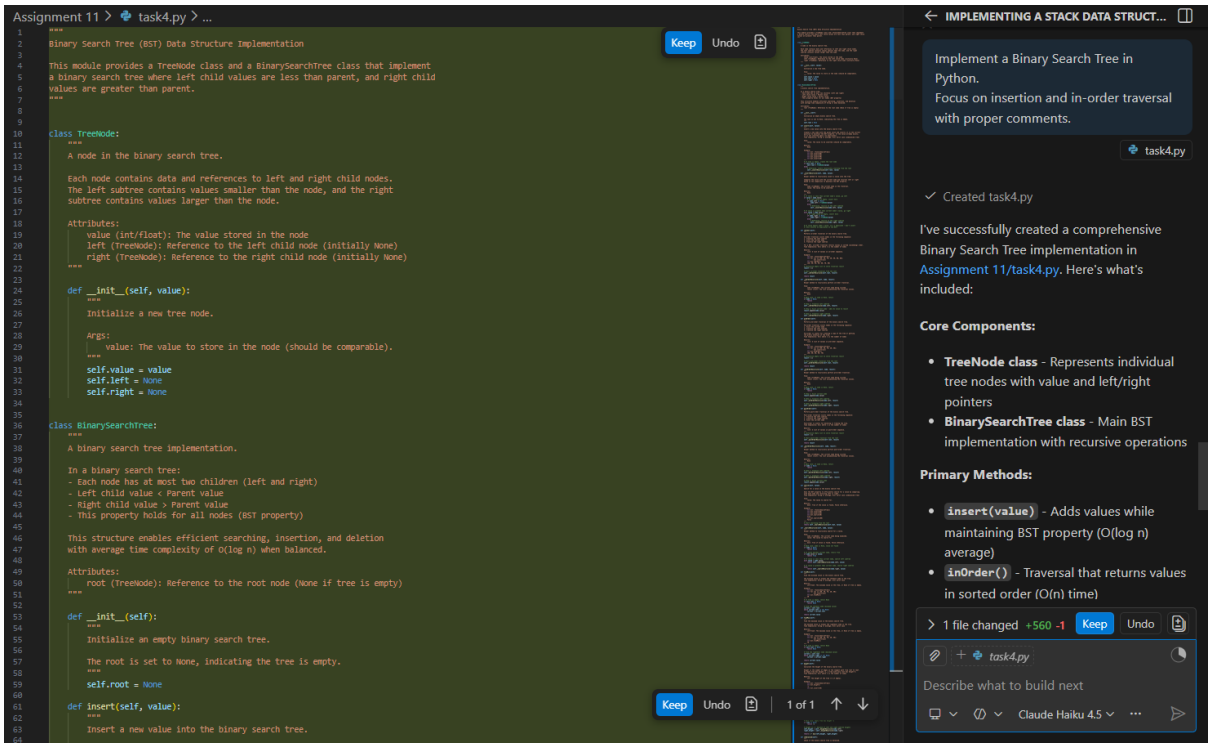
```
Traversing the list:
Values: [5, 10, 20, 25, 30, 40, 50]

Search operations:
Search for 30: True
Search for 100: False

Deleting node with value 25:
Linked List: 5 -> 10 -> 20 -> 30 -> 40 -> 50 -> None
Length: 6

Reversing the linked list:
Linked List: 50 -> 40 -> 30 -> 20 -> 10 -> 5 -> None
Length: 6

=== Real-World Example: Student Records ===

Adding students: ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve']
Linked List: Alice -> Bob -> Charlie -> Diana -> Eve -> None
Total students: 5

Is 'Bob' enrolled? True
Is 'Frank' enrolled? False

Removing 'Charlie' from the list:
Linked List: Alice -> Bob -> Diana -> Eve -> None
Total students: 4
```

Task Description -4 – (Binary Search Tree Operations)

• Task: Implement a Binary Search Tree with AI support focusing on insertion and traversal.

Expected Output:

• BST program with correct node insertion and in-order traversal output.



OUTPUT:

```
Inserting values: [50, 30, 70, 20, 40, 60, 80, 10, 25, 35, 65]

In-Order Traversal (sorted):
[10, 20, 25, 30, 35, 40, 50, 60, 65, 70, 80]

Pre-Order Traversal:
[50, 30, 20, 10, 25, 40, 35, 70, 60, 65, 80]

Post-Order Traversal:
[10, 25, 20, 35, 40, 30, 65, 60, 80, 70, 50]

Search Operations:
Search for 40: True
Search for 100: False
Search for 25: True

Minimum value: 10
Maximum value: 80

Tree height: 3
Is tree balanced? True

=== Real-World Example: Employee ID Database ===

Adding employee IDs: [5001, 5020, 5010, 5005, 5015, 5025, 5030, 5008, 5012]
Employees in sorted order: [5001, 5005, 5008, 5010, 5012, 5015, 5020, 5025, 5030]

Employee ID 5015 found in database

ID Range: 5001 to 5030
```

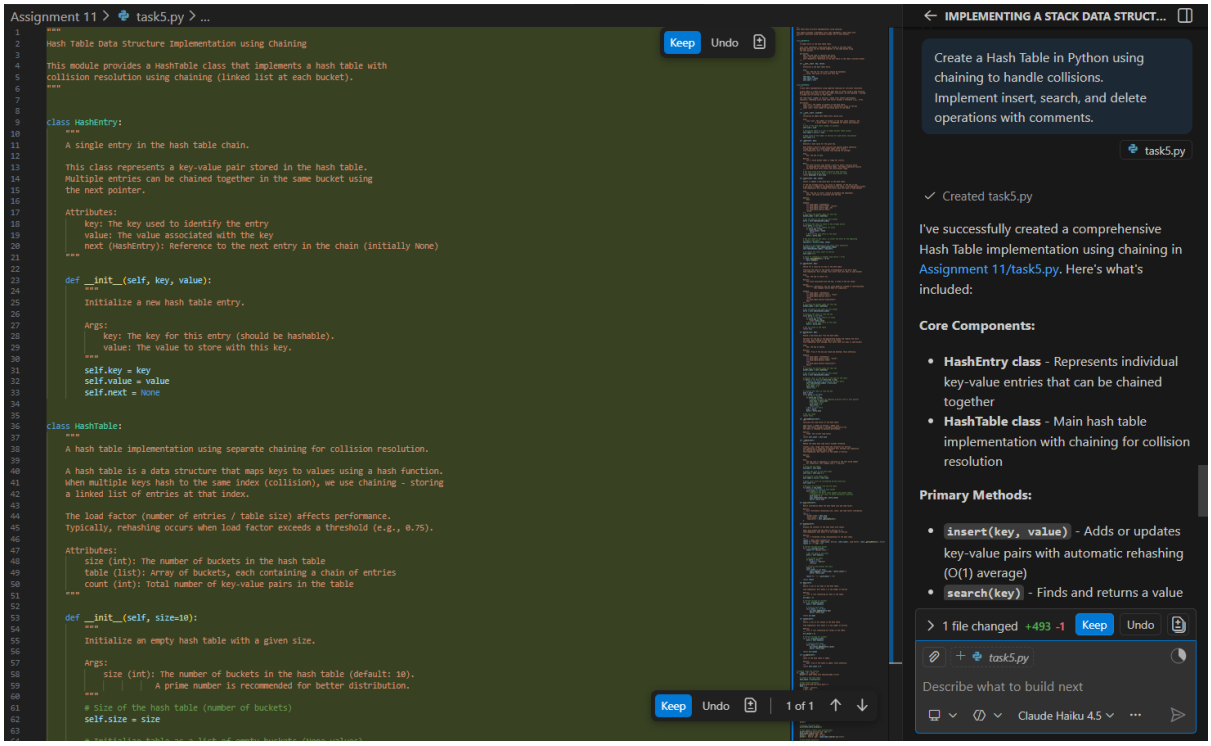
Task Description -5 – (Hash Table Implementation)

- **Task: Create a hash table using AI with collision handling**

Expected Output:

- **Hash table supporting insert, search, and delete using chaining**

or open



OUTPUT:

```
Hash Table Contents:
Buckets: 20, Entries: 10, Load Factor: 0.50
--------------------------------------------------
Bucket [0]: Empty
Bucket [1]: Empty
Bucket [2]: (Diana: 88)
Bucket [3]: Empty
Bucket [4]: Empty
Bucket [5]: Empty
Bucket [6]: Empty
Bucket [7]: (Eve: 90)
Bucket [8]: Empty
Bucket [9]: Empty
Bucket [10]: (Charlie: 92)
Bucket [11]: (Alice: 95)
Bucket [12]: Empty
Bucket [13]: Empty
Bucket [14]: (Bob: 87)
Bucket [15]: (Grace: 96)
Bucket [16]: Empty
Bucket [17]: Empty
Bucket [18]: Empty
Bucket [19]: Empty
```